

Dhwani - Simple Project and Technology Guide

A short, plain-language explanation of the Android app and its complete technology stack.

What is Dhwani?

Dhwani is an Android call-assistance app for deaf and hard-of-hearing people. It works beside a normal speakerphone call. The app converts the caller's speech into readable captions and converts the user's written, suggested, or signed reply into spoken audio.

The main idea

1	The caller speaks The call is placed on speaker so the phone microphone can hear the caller.
2	Dhwani creates captions An offline speech model changes the sound into English or Hindi text.
3	The user chooses a reply The reply can be typed, selected from suggestions, or created from a recognized sign.
4	The phone speaks Android text-to-speech reads the chosen reply aloud to the caller.

Main features

- Offline English and Hindi live captions.
- Typed replies spoken through the phone.
- Three short smart-reply suggestions from an optional on-device Gemma model.
- Outgoing-call preparation and local call summaries.
- Front-camera recognition for a focused set of Indian Sign Language signs.
- Encrypted storage for profile details and call history.

Important

The sign models are included in the repository. The English/Hindi caption folders and Gemma model require additional setup before those features are fully usable.

1. How a person uses Dhwani

First-time setup

The first screen introduces Dhwani and saves a small flag after onboarding is completed. The app then requests microphone and camera permissions. The microphone is needed for captions, while the camera is needed for sign recognition.

The five main tabs

Tab	Purpose	What the person can do
Home	Starting point	Open Live, Sign, Call, or profile features
Live	Follow a call	Choose language, start captions, read speech, and reply
Sign	Use ISL input	Show one sign to the front camera and prepare a sentence
Call	Prepare a call	Enter a goal, get a briefing, enter a number, and open the dialer
You	Save personal details	Set name, language, address, contacts, and other safe context

Typical call flow

1	Prepare Optionally save useful details and enter the reason for the call.
2	Open the dialer Dhwani fills the number in Android's dialer. The user confirms the call.
3	Use speakerphone The phone call must be audible to the microphone.
4	Start captions Choose English or Hindi, then tap Start captions.
5	Reply Type, tap a suggestion, or recognize a supported sign. Review the sentence.
6	Speak Tap Speak so Android reads the sentence aloud.
7	Finish Stop captions. If Gemma is available, a short encrypted summary can be saved.

What Dhwani does not do

- It does not replace the phone app or telephone network.
- It does not directly capture protected internal call audio; it listens through speakerphone.
- It does not automatically place a call. It opens the dialer for user confirmation.
- It is not a complete continuous ISL sentence translator.

2. Live captions and spoken replies

How live captions work

When Start captions is tapped, Dhvani prepares the selected Vosk speech model, changes the phone to communication audio mode, switches on speakerphone, starts a visible foreground service, and begins recording through the microphone.

Stage	What happens	Technology
Audio capture	The microphone records 16 kHz, mono, 16-bit sound	Android AudioRecord
Audio cleanup	Echo cancellation and noise suppression are used when the phone supports them	Android audio effects
Recognition	Small sound chunks are converted into partial or final words	Vosk
Display	Changing partial text is replaced; final text becomes a fixed transcript line	StateFlow + Compose
Background activity	An ongoing notification shows that captioning is active	Foreground service

English and Hindi

The user chooses English or Hindi before captioning begins. The code expects complete Vosk models in folders named **vosk-en** and **vosk-hi**. Language cannot be changed during an active caption session.

Spoken replies

The user can type any reply or select an existing sentence. Dhvani adds the reply to the transcript and sends it to Android's text-to-speech engine. Text containing Devanagari characters uses hi-IN; other text uses en-IN.

Why speakerphone is necessary

Android normally protects call audio. Dhvani therefore listens to sound played by the phone speaker. Caption accuracy depends on speaker volume, distance, background noise, echo, accent, and the quality of the installed speech model.

Current model issue

The repository's vosk-en and vosk-hi folders are placeholders. A full Hindi model exists under vosk-model-small-hi-0.22, but the app does not use that folder name. The model must be moved or the code mapping must be changed.

3. Smart replies, call preparation, and summaries

What the optional Gemma model does

Feature	Result	Information used
Smart replies	Exactly three short replies, normally under ten words	Latest caller sentence, recent transcript, and selected local facts
Call briefing	Opening sentence, likely questions, possible replies, and useful details	Call goal, profile, contacts, and recent summaries
Call summary	Two short lines describing the outcome and follow-up	Up to the latest 30 completed transcript lines

How local fact lookup works

Gemma can ask the app for a specific saved fact. The request is parsed by Dhvani, looked up in encrypted local records, and returned to the model. The model is not given uncontrolled access to the phone.

Local lookup	What it returns
get_address	A saved address and landmark
get_contact_info	A person's role, number, notes, and last-call information
get_medical_info	Medicines, allergies, conditions, or emergency contact
get_payment_hint	Only a user-written safe-to-share payment hint
get_recent_call_summary	Recent summaries, optionally filtered by contact

Model details

- Expected file: gemma3-1b-it-int4.task.
- Runtime: MediaPipe Tasks GenAI LLM Inference.
- Maximum output/context setting in code: 2,048 tokens; top-k up to 40.
- The model is loaded only when first needed and must be at least 100 MB.
- Only one generation runs at a time, reducing memory and concurrency problems.
- The repository does not include this Gemma file.

Fallback

If Gemma is unavailable, typed replies still work. Dhvani can also show three fixed replies: 'Yes, please continue.', 'Can you repeat that?', and 'I will get back to you soon.'

4. Indian Sign Language recognition

The Sign tab uses the front camera to recognize one isolated sign. The user taps Recognize and performs one complete sign during a 3.6-second capture.

Recognition process

1	Capture CameraX supplies frames from the front camera.
2	Find body points MediaPipe finds 25 pose points and 21 points on each hand.
3	Build model input The app stores 134 x/y values per frame and fills short gaps.
4	Run two recognizers A MediaPipe model checks general gestures; the INCLUDE Transformer checks ISL words.
5	Check reliability Confidence, framing, motion, vocabulary, and time agreement are tested.
6	Create reply The accepted sign is changed into a short English or Hindi sentence.

Models

Model	Purpose
Holistic Landmarker	Finds upper-body and hand landmarks
Gesture Recognizer	Recognizes seven general gestures such as thumbs up
AI4Bharat INCLUDE Transformer	Classifies 263 isolated ISL word classes from movement over time

Safety checks

- At least 12 model frames and 8 hand frames are required.
- Shoulders, an elbow, and enough hand points must be visible for reliable framing.
- The sign is checked in up to three time views: full, start-trimmed, and end-trimmed.
- At least two of three views must agree.
- The best answer must be sufficiently stronger than the second-best answer.
- Unsupported or uncertain results return Unknown instead of a forced word.

Scope

This is isolated-sign recognition, not continuous sentence translation. It works best when the face, shoulders, elbows, and hands are visible and one supported sign is performed once.

5. Saved data, privacy, and permissions

Information the user can save

Information	Examples	Reason
Profile	Name and language	Personalized replies
Addresses	Home address, landmark, simple spoken form	Quick address answers
Contacts	Name, role, numbers, notes	Call preparation
Medical context	Medicines, allergies, conditions, emergency contact	Health-related calls
Payment hints	Only details marked safe to share	Billing conversations
Call history	Transcript, summary, duration, contact, outcomes	Remember previous calls

Encryption

Profile data and call history are stored as JSON after being encrypted with AES-GCM. The secret key is created and protected by Android Keystore. A new random initialization value is used for each saved item. This protects confidentiality and helps detect modified encrypted data.

Permissions

Permission	Why it is required
RECORD_AUDIO	Listen to speakerphone sound for captions
CAMERA	Recognize ISL signs
FOREGROUND_SERVICE	Keep an active caption session running visibly
FOREGROUND_SERVICE_MICROPHONE	Follow newer Android microphone-service rules
POST_NOTIFICATIONS	Show the active caption notification
MODIFY_AUDIO_SETTINGS	Use communication mode and speakerphone

Privacy facts

- The manifest does not request internet access.
- It does not request phone contacts, call logs, direct-call permission, or storage access.
- Saved contacts are entered inside Dhwani; they are not copied from the phone address book.
- The app currently sets `allowBackup=true`. Production backup and restore behavior should be reviewed.
- Transcription laws differ by location. Users may need to inform the other caller.

6. Software architecture and project structure

Main software layers

Layer	Main components	Responsibility
User interface	Compose screens, theme, navigation	Shows captions, controls, forms, camera, and status
Coordinator	CallViewModel, StateFlow, coroutines	Connects user actions to every feature
Audio	Recorder, Vosk wrapper, TTS, CallService	Captures speech, creates captions, and speaks replies
Local intelligence	Gemma, tool dispatcher, sign recognizers	Writes suggestions and recognizes signs
Data	UserContext, CallLogStore, SecureJsonStore	Parses, encrypts, saves, loads, and clears records
Android system	Camera, audio, dialer, Keystore, notifications	Provides protected device capabilities

Central coordinator

CallViewModel holds the current screen state and coordinates captions, speech, sign capture, Gemma generation, call preparation, and saved summaries. Kotlin StateFlow sends state changes to Jetpack Compose so the interface updates automatically.

Important folders

Folder	Contents
app/src/main/java/.../ui	Screens, navigation, theme, and CallViewModel
app/src/main/java/.../audio	Recorder, Vosk, and TTS
app/src/main/java/.../sign	Camera analysis, ONNX model, policy, and vocabulary
app/src/main/java/.../llm	Gemma runtime, prompts, and local tool lookup
app/src/main/java/.../data	Profile, call history, onboarding, and encryption
app/src/main/assets	Speech and sign model assets
tools/sign	INCLUDE conversion and video evaluation
training/sign	Experimental Gemma vision LoRA training
app/src/test and androidTest	Unit tests and an on-device model smoke test

Background work

Kotlin coroutines keep model loading, speech processing, suggestions, summaries, and TTS away from the main screen thread. Separate jobs can be cancelled when the user stops a call or starts a newer task.

7. Complete technology stack

Technology	Version or setting	Simple purpose
Android	compile 36; target 34; minimum 26	Mobile platform
Kotlin	2.0.0; Java/JVM 17	Main programming language
Android Gradle Plugin	8.13.2	Builds and packages the app
Jetpack Compose	BOM 2024.05.00; Material 3	Creates the user interface
AndroidX Activity/Core	1.9.0 / 1.13.1	Modern Android foundation
Android Lifecycle	2.8.0	Manages screen state and cleanup
Kotlin Coroutines	1.8.1	Runs background work
CameraX	1.3.3	Front-camera preview and frame analysis
Accompanist Permissions	0.34.0	Permission handling
Vosk Android	0.3.75	Offline speech-to-text
JNA	5.19.1	Native bridge used by speech libraries
Android TextToSpeech	System service	Reads replies aloud
MediaPipe Tasks GenAI	0.10.27	Runs Gemma on the phone
MediaPipe Tasks Vision	0.10.26.1	Tracks body/hands and general gestures
ONNX Runtime Android	1.24.3	Runs the INCLUDE sign model
Protocol Buffers	4.26.1	Model/runtime data dependency
Android Keystore	System service	Protects encryption keys
AES-GCM	128-bit authentication tag	Encrypts saved data
JUnit	4.13.2	Local unit testing
AndroidX Test	1.2.1 / 1.6.2	On-device tests
Python + PyTorch	Development scripts	Model conversion, training, and evaluation
OpenCV + MediaPipe	Development scripts	Reads videos and extracts landmarks
Unsloth + TRL	Experimental training	Efficient Gemma vision LoRA fine-tuning

Build target

The app packages ARM64 native libraries only. It is intended for Android 8.0 or newer phones with enough storage and memory for local models.

8. Models, testing, and requirements

Model files

Model or asset	Status in repository	Purpose
gesture_recognizer.task	Included	Seven general hand gestures
holistic_landmarker.task	Included	Body and hand landmarks
include_transformer_small.onnx	Included	263-class isolated ISL recognition
include_transformer_labels.json	Included	Readable sign labels
gemma3-1b-it-int4.task	Not included	Smart replies, briefings, summaries
vosk-en	Placeholder	English captions
vosk-hi	Placeholder	Hindi captions
vosk-model-small-hi-0.22	Files included under different name	Hindi speech model

Device and development requirements

- Android Studio, JDK 17, Android SDK 36, and Gradle sync.
- Android 8.0+ ARM64 phone with USB debugging for development installation.
- Microphone and speakerphone for live captions; front camera for sign input.
- Installed en-IN or hi-IN Android speech data for spoken replies.
- Enough storage and RAM for the selected Vosk and Gemma models.

Tests already present

Test area	What is checked
Captions	Partial and final transcript merging
Speech models	Required Vosk files exist and are non-empty
Sign preprocessing	Scaling, padding, missing points, frame rate, and timing views
Sign safety policy	Confidence, ambiguity, unsupported signs, and weak framing
Sign vocabulary	English/Hindi phrases and saved address use
Assistant tools	Local tool-call parsing and suggestion cleanup
Android smoke test	Bundled sign models load and run on a device

Sign development utilities

Python scripts convert the official AI4Bharat INCLUDE checkpoint to ONNX, compare ONNX and PyTorch outputs, evaluate real videos, simulate lower phone frame rates, and calculate top-1, top-5, and per-category accuracy. A separate experimental script fine-tunes a Gemma 4 vision model with LoRA.

9. Current limitations and setup

Important current limitations

Limitation	Effect	Suggested fix
Vosk folders are placeholders	Captions cannot start in a fresh checkout	Package verified models in vosk-en and vosk-hi
Hindi folder name mismatch	Existing Hindi files are not found	Rename the asset or update the mapping
Gemma is missing	No generated briefing, replies, or summary	Add the correct task model and document its license/checksum
Speakerphone listening	Noise and echo may reduce accuracy	Test several phones and provide audio guidance
One sign at a time	No continuous ISL sentences	Keep scope clear and expand only after measured testing
Small accepted sign set	Many signs return Unknown	Add classes only with accuracy and confusion evidence
Permissions requested together	Camera is requested before Sign is opened	Request each permission only when needed
ARM64 only	Other processor types cannot install	Add only tested architectures
allowBackup=true	Encrypted restore behavior may be unclear	Define and test backup rules

Basic setup

1	Open the project Open D:\dhwani in Android Studio and allow Gradle to sync.
2	Check Android SDK Android Studio should create local.properties pointing to SDK 36.
3	Add speech models Place complete English and Hindi Vosk models in the exact expected asset folders.
4	Add Gemma if needed Place gemma3-1b-it-int4.task in app/src/main/assets/models.
5	Connect phone Use an Android 8.0+ ARM64 phone and enable USB debugging.
6	Install Run .\gradlew.bat installDebug on Windows.
7	Use Put a call on speaker, open Live, start captions, and choose a reply.

Final summary

Dhwani combines offline speech recognition, Android text-to-speech, a local language model, camera-based ISL recognition, and encrypted storage in one accessibility-focused app. The main architecture is already present. The most important next work is correct model packaging, real-device testing, caption/sign accuracy measurement, and accessibility review with intended users.

Repository snapshot

This guide describes branch main at commit fa36056 and application version 0.1.0. Repository: <https://github.com/Anant-0705/Svara>